`run_egmn.py` (Q&A Documentation)

Q: What does this run script control? **A:** It controls the full experiment loop: args, data loading, model instantiation, training with a specific loss/optimizer, and evaluation with shared metrics.

Q: Why is the model not responsible for training/evaluation? **A:** Keeping training logic in the script makes it easy to compare models under the same protocol and avoids mixing I/O/metrics with differentiable code.

Q: What does this file export (public API)? **A:** The public API is the set of classes/functions other modules import (sometimes re-exported by package initializers). The key top-level definitions detected here are:

```
def get_args(...)  # function
def get_loaders(...)  # function
def mae_rescale_to_second(...)  # function
def test(...)  # function
```

Q: What are the main dependencies (imports) and why do they matter? **A:** Imports show whether this file is model code (torch), data code (pandas), or evaluation/analysis (sklearn). They also determine runtime requirements.

```
import os
import copy
import torch
import random
import numpy as np
import argparse
from dataloader import KUAIRECDataLoader
from model import EGMN
from utils import eval_mae, eval_xauc, eval_kl
from sklearn.metrics import roc_auc_score

def get_args():
    parser = argparse.ArgumentParser()
    parser.add_argument('--dataset_name', default='kuairec')
    parser.add_argument('--dataset_path', default='./dataset/')
    parser.add_argument('--device', default='cuda:0')
    parser.add_argument('--bsz', type=int, default=2048)
    parser.add_argument('--log_interval', type=int, default=10)

    parser.add_argument('--alpha', type=float, default=0.1)
    parser.add_argument('--beta', type=float, default=1.0)
```

```
    parser.add_argument('--epoch', type=int, default=10)
    parser.add_argument('--lr', type=float, default=0.1)
    parser.add_argument('--weight_decay', type=float, default=1e-6)
    parser.add_argument('--runs', type=int, default=1, help = 'number of executions to compute the avera
    parser.add_argument('--seed', type=int, default=42)

    args = parser.parse_args()
    return args

def get_loaders(name, dataset_path, device, bsz):
    path = os.path.join(dataset_path, name, "{}_data.pkl".format(name))
    if name == 'kuairec':
        dataloaders = KUAIRECDataLoader(name, path, device, bsz=bsz)
    else:
        raise ValueError('unkown dataset name: {}'.format(name))
    return dataloaders

def mae_rescale_to_second(dataset, mae):
    if dataset == 'kuairec':
        return mae * 999639 / 1000
    elif dataset == 'wechat':
        return mae * 20840
    elif dataset == 'cikm16':
        return (mae *(6000-31) + 31) / 1000
    else:
        raise ValueError('unkown dataset name: {}'.format(dataset))

def test(args, model, dataloaders):
    model.eval()
```

Q: What is `get_args` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `get_args`? **A:** Excerpt from the file:

```
def get_args():
    parser = argparse.ArgumentParser()
    parser.add_argument('--dataset_name', default='kuairec')
    parser.add_argument('--dataset_path', default='./dataset/')
    parser.add_argument('--device', default='cuda:0')
    parser.add_argument('--bsz', type=int, default=2048)
    parser.add_argument('--log_interval', type=int, default=10)

    parser.add_argument('--alpha', type=float, default=0.1)
    parser.add_argument('--beta', type=float, default=1.0)
    parser.add_argument('--epoch', type=int, default=10)
    parser.add_argument('--lr', type=float, default=0.1)
    parser.add_argument('--weight_decay', type=float, default=1e-6)
    parser.add_argument('--runs', type=int, default=1, help = 'number of executions to compute the average metrics')
    parser.add_argument('--seed', type=int, default=42)

    args = parser.parse_args()
    return args
```

Q: What is `get_loaders` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `get_loaders`? **A:** Excerpt from the file:

```
def get_loaders(name, dataset_path, device, bsz):
    path = os.path.join(dataset_path, name, "{}_data.pkl".format(name))
    if name == 'kuairec':
        dataloaders = KUAIRECDataLoader(name, path, device, bsz=bsz)
    else:
        raise ValueError('unkown dataset name: {}'.format(name))
    return dataloaders
```

Q: What is `mae_rescale_to_second` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `mae_rescale_to_second`? **A:** Excerpt from the file:

```
def mae_rescale_to_second(dataset, mae):
    if dataset == 'kuairec':
        return mae * 999639 / 1000
    elif dataset == 'wechat':
        return mae * 20840
    elif dataset == 'cikm16':
        return (mae *(6000-31) + 31) / 1000
    else:
        raise ValueError('unkown dataset name: {}'.format(dataset))
```

Q: What is `test` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `test`? **A:** Excerpt from the file:

```
def test(args, model, dataloaders):
    model.eval()
    labels, scores, predicts = list(), list(), list()
    with torch.no_grad():
        for _, (features, label) in enumerate(dataloaders['test']):
            y = model.predict(features)
            duration = features['duration'].squeeze()
            pred = y.squeeze()
            labels.extend(label.tolist())
            scores.extend(pred.tolist())
    labels, scores = np.array(labels), np.array(scores)
    mae, xauc, kl = eval_mae(labels, scores), eval_xauc(labels, scores), eval_kl(labels, scores)
    mae = mae_rescale_to_second(args.dataset_name, mae)
    print("test result | MAE: {:.7f} | XAUC: {:.7f} | KL: {:.7f}".format(mae, xauc, kl))
```

Q: Example: end-to-end data flow involving this file **A:** Core pipeline shape (schematic):

```
python run_egmn.py --dataset_name kuairec --dataset_path ./dataset --device cuda:0
# Internally: load dataloaders → build model → train → evaluate
```

Q: Full source code of the Python file **A:** The complete file is included verbatim for reference.

```
import os
import copy
import torch
import random
```

```
import numpy as np
import argparse
from dataloader import KUAIRECDataLoader
from model import EGMN
from utils import eval_mae, eval_xauc, eval_kl
from sklearn.metrics import roc_auc_score

def get_args():
    parser = argparse.ArgumentParser()
    parser.add_argument('--dataset_name', default='kuairec')
    parser.add_argument('--dataset_path', default='./dataset/')
    parser.add_argument('--device', default='cuda:0')
    parser.add_argument('--bsz', type=int, default=2048)
    parser.add_argument('--log_interval', type=int, default=10)

    parser.add_argument('--alpha', type=float, default=0.1)
    parser.add_argument('--beta', type=float, default=1.0)
    parser.add_argument('--epoch', type=int, default=10)
    parser.add_argument('--lr', type=float, default=0.1)
    parser.add_argument('--weight_decay', type=float, default=1e-6)
    parser.add_argument('--runs', type=int, default=1, help = 'number of executions to compute the average metrics')
    parser.add_argument('--seed', type=int, default=42)

    args = parser.parse_args()
    return args

def get_loaders(name, dataset_path, device, bsz):
    path = os.path.join(dataset_path, name, "{}_data.pkl".format(name))
    if name == 'kuairec':
        dataloaders = KUAIRECDataLoader(name, path, device, bsz=bsz)
    else:
        raise ValueError('unkown dataset name: {}'.format(name))
    return dataloaders

def mae_rescale_to_second(dataset, mae):
    if dataset == 'kuairec':
        return mae * 999639 / 1000
    elif dataset == 'wechat':
        return mae * 20840
    elif dataset == 'cikm16':
        return (mae *(6000-31) + 31) / 1000
    else:
        raise ValueError('unkown dataset name: {}'.format(dataset))

def test(args, model, dataloaders):
    model.eval()
    labels, scores, predicts = list(), list(), list()
    with torch.no_grad():
        for _, (features, label) in enumerate(dataloaders['test']):
            y = model.predict(features)
            duration = features['duration'].squeeze()
            pred = y.squeeze()
            labels.extend(label.tolist())
            scores.extend(pred.tolist())
    labels, scores = np.array(labels), np.array(scores)
    mae, xauc, kl = eval_mae(labels, scores), eval_xauc(labels, scores), eval_kl(labels, scores)
    mae = mae_rescale_to_second(args.dataset_name, mae)
    print("test result | MAE: {:.7f} | XAUC: {:.7f} | KL: {:.7f}".format(mae, xauc, kl))

if __name__ == '__main__':
    args = get_args()
    if args.seed > -1:
        np.random.seed(args.seed)
        torch.manual_seed(args.seed)
        torch.cuda.manual_seed(args.seed)
    res = {}
    torch.cuda.empty_cache()
```

```
device = torch.device(args.device)

dataloaders = get_loaders(args.dataset_name, args.dataset_path, device, args.bsz)
model = EGMN(dataloaders.description, embed_dim=16, share_mlp_dims=(256, 128, 64), output_mlp_dims=(32, 16), dropout=0.2)
model = model.to(device)
model.train()

# train
dataloader_train = dataloaders['train']
# optimizer = torch.optim.Adam(params=model.parameters(), lr=args.lr, weight_decay=args.weight_decay)
optimizer = torch.optim.Adagrad(params=model.parameters(), lr=args.lr, weight_decay=args.weight_decay)
for epoch_i in range(1, args.epoch + 1):
    model.train()
    epoch_loss, epoch_nll, epoch_reg, epoch_entropy = 0.0, 0.0, 0.0, 0.0
    total_loss, total_nll, total_reg, total_entropy = 0.0, 0.0, 0.0, 0.0
    total_iters = len(dataloader_train)
    for i, (features, label) in enumerate(dataloader_train):
        pi, lambda_, mu, sigma = model(features)
        duration = features['duration'].squeeze()
        nll_loss, reg_loss, entropy_loss = model.loss(label.float(), pi, lambda_, mu, sigma, features['duration'].view(-1, 1
        loss = nll_loss + args.alpha * entropy_loss + args.beta * reg_loss

        model.zero_grad()
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item(); epoch_nll += nll_loss.item(); epoch_reg += reg_loss.item(); epoch_entropy += entropy_loss
        total_loss += loss.item(); total_nll += nll_loss.item(); total_reg += reg_loss.item(); total_entropy += entropy_loss
        if (i + 1) % 10 == 0:
            print("    Iter {}/{} loss: {:.7f}, epoch nll: {:.7f}, epoch reg: {:.7f}, epoch entropy: {:.7f}".format(i + 1, t
            total_loss, total_nll, total_reg = 0, 0, 0
    print("Epoch {}/{} average Loss: {:.7f}, nll: {:.7f}, reg: {:.7f}, entropy: {:.7f}".format(epoch_i, args.epoch, epoch_lo
test(args, model, dataloaders)
```